

Getting Started With Quarto

Josh Lim

2026-08-01

Table of contents

About this resource	5
1 What is Quarto?	6
1.1 Why Use Quarto?	6
1.1.1 Write Once, Publish Anywhere	6
1.1.2 Accessible formats	6
1.1.3 Plain Text Source Files	7
1.1.4 Rich content	7
1.2 The anatomy of a Quarto project	7
1.3 The Quarto stack	8
2 Installing the required Software	9
2.1 1. Install Quarto	9
2.2 Install TeX	9
2.3 Install Visual Studio Code	10
3 Creating Your First Quarto Document	11
4 Setting up your Quarto project	12
4.1 Rendering your project	12
4.1.1 Rendering from VSCode	12
4.1.2 Rendering from the command line	13
4.1.3 Quick preview	13
5 Basic Markdown	14
5.1 Headings	14
5.2 Bold and Italics	14
5.3 Lists	14
5.4 Links	15
5.5 Basic Images	15
6 Converting Existing LaTeX Notes	17
6.1 Recommended Migration Approach	17
6.2 Automatically converting LaTeX	18
6.3 Tidying up the .qmd file	18

7	Figures	19
7.1	Basic Figure	19
7.2	Layout Options	21
7.2.1	Size	21
7.2.2	Subfigures	21
7.2.3	Two Images Above One Wide Image	22
7.2.4	Unequal Widths	23
7.2.5	Adding Space Between Elements	23
7.3	Figures from code	23
7.4	Figure Divs	24
7.5	Tikz figures	24
8	Maths Equations	25
8.1	Inline Mathematics	25
8.2	Display Mathematics	25
8.3	Numbered Equations	25
8.4	Aligned equations	26
8.5	Referencing Equations	27
8.6	Newcommands and macros	27
9	Code	28
9.1	Including inline code snippets	28
9.2	Including code blocks	28
9.3	Running Code	29
9.4	Options for running code	29
9.4.1	Showing Output but Hiding Code	29
9.4.2	Folding Long Code Blocks	30
9.5	Figures from code	30
9.6	Global Code Options	32
10	Working with Sections	34
10.1	How sections are displayed	34
10.2	Numbering sections	34
10.2.1	Global defaults	35
10.3	Labelling sections	35
10.4	Referencing sections	35
11	References and Citations	36
11.1	Connecting the Bibliography	36
11.2	Reference list	36

12 More Markdown	38
12.1 Tables	38
12.1.1 Labelled Tables	38
12.2 Collapsible Sections	39
12.3 Callout highlighted Boxes	39
12.4 Footnotes	40
12.4.1 Inline footnotes	40
12.4.2 Defining footnotes	41
12.5 Hiding/commenting out content	41
12.6 Quarto extensions	42
13 Tweaking the YAML Header	43
13.1 Table of Contents	43
13.2 Section Numbering	43
13.3 Themes	43
14 Multiple Output Formats	45
15 Creating Books and Lecture Notes	46
16 Publishing a Quarto Project	48
16.1 Publish to Moodle	48
16.1.1 Moodle publishing workflow	48
16.1.2 Updating a Moodle-hosted site	49
16.2 Public Resources: GitHub Pages	49
References	50
I Appendices	51

About this resource

This is a Quarto book is for academic staff who want to get started using Quarto. We cover:

- basic installation and building your first Quarto project
- tips for migrating from a LaTeX source
- using Quarto with a focus on lecture note things like: maths, figures, referencing and much more!

Tip

If you prefer, the [Quarto website](#) has a more complete set of information and examples: the purpose of this guide is to provide a more focused and practical introduction to Quarto for academic staff.

Note

This guide introduces the fundamentals of Quarto using a **Visual Studio Code (VS Code) workflow**.

Alternatively you can also use Quarto through RStudio (e.g. see [Quarto for Scientists - an R Studio Workflow](#)), the command line and other routes too!

Feedback?

If you have any feedback or suggestion on how to make this book more useful then please get in touch with [Josh Lim](#) - University of Bath.

In particular I'd welcome:

- Errors or corrections
- Calls for clarification
- Missing info
- Suggestions for improvements/new topics

1 What is Quarto?

Quarto is a modern publishing system that allows you to create teaching materials from simple plain-text source files and publish them to a range of formats including:

- HTML websites
- PDF documents
- Microsoft Word documents
- PowerPoint presentations
- Reveal.js slide decks
- Books and collections of lecture notes

Note

This guide introduces the fundamentals of Quarto using a **Visual Studio Code (VS Code) workflow**.

Alternatively you can also use Quarto through RStudio, the command line and other routes too!

1.1 Why Use Quarto?

1.1.1 Write Once, Publish Anywhere

A single source document can be rendered as:

- A website for online reading
- A printable PDF
- A Word document
- Presentation slides

1.1.2 Accessible formats

HTML output can provide:

- Responsive design

- Screen reader support
- Improved navigation
- Adjustable text size
- Better support for mobile devices (compared to PDFs)

1.1.3 Plain Text Source Files

Quarto files are plain text, making them:

- Easy to edit
- Easy to archive
- Suitable for version control systems such as Git
- Portable across operating systems

Note

If you're from a LaTeX background or bookdown then you'll find the equation syntax and referencing familiar (citations can be managed with a .bib file). However, given how extensible LaTeX is, you won't have access to all the packages you may be familiar with.

1.1.4 Rich content

You can:

- embed content (e.g. video, interactive widgets)
- execute code and render outputs (e.g. Python, R)

1.2 The anatomy of a Quarto project

A typical Quarto project consists of:

- A project folder
- A `_quarto.yml` configuration file that sets project-level settings (e.g. formatting options, output formats). Similar to a LaTeX preamble, it's useful to have this separately from the source files so you can easily re-use it.
- One or more Quarto source files (with the extension `.qmd`). This is written in markdown and is the main bulk of your document.
- a folder for images and other resources

Optionally, you may also include: - a bibliography file (with the extension `.bib`) for citations - a folder for custom scripts (e.g. Python, R) that can be executed during the rendering process - a folder for custom extensions (e.g. for new output formats or features - see [Quarto extensions](#))

1.3 The Quarto stack

Quarto helps automate the process of converting your source content and configuration into a range of publication formats. You write your content in a `.qmd` file and control how it is rendered using YAML settings (either within the document or in project-level `.yaml` files). Quarto then takes care of the remaining steps, so you can focus on writing content rather than managing complex publishing workflows.

Rendering workflow

1. **You write content** in a `.qmd` file using Markdown, equations, images, and optional code (e.g. additional `.py`, `.R`, `.ipynb` files).
2. **You define settings** using YAML metadata (your `_quarto.yaml` file or a YAML block within the document).
3. **Quarto executes code** (if present) using **Knitr**, **Jupyter**, or another supported engine.
4. **Quarto converts content** using Pandoc.
5. **Quarto generates final output** in one or more formats. This includes HTML books (*like this one*), PDF documents (via LaTeX) and other formats (e.g. Word, Reveal.js, EPUB)

In practice, most authors only need to maintain their `.qmd` files and YAML settings (steps 1 and 2 above). Quarto automates the execution, conversion, and creation steps, allowing you to create professional outputs from a single source document.

2 Installing the required Software

A typical Quarto workflow requires:

1. Quarto (this is a command line tool to render the outputs)
2. a TeX installation (for PDF output)
3. Visual Studio Code (VS Code)

Additionally if you wish to execute python/R code from your Quarto project, you should this installed.

Note

This is similar to LaTeX, where you'd have TeXLive/MikTeX installation and then a editor of your choice

2.1 1. Install Quarto

Download Quarto from: <https://quarto.org>

After installation, to check Quarto has been installed and added into the Path, open a terminal and run:

```
quarto check
```

2.2 Install TeX

This is not covered here, but there are lots of ways of getting TeX!

A minimal installation, where you can add additional packages as you go may be enough!

2.3 Install Visual Studio Code

Download VS Code from: <https://code.visualstudio.com>

Recommended extensions:

- Quarto
- Markdown All in One

Some other noteworthy extensions for writing documents in VSCode:

- GitLens
- Zotero Citation Picker

3 Creating Your First Quarto Document

4 Setting up your Quarto project

1. Create a new directory for your project.
2. Launch VSCode
3. Open the command palette (Ctrl-shift-P) and begin typing `Quarto: Create Project`
4. Select “Book Project”
5. Choose the Project directory you created in step 1.

This creates:

- a simple `_quarto.yml` file (effectively your preamble)
- a few chapters (`*.qmd`)
- a `references.bib` (a simple bibtex file)

Tip

You may find it useful to create a **figures** and **chapters** folder to keep your top level directory tidy.

Note

In this demo we use “Book Project” as the starting point. Alternatively, you may choose a “Default Project” (which has fewer default files) or any of the other templates.

4.1 Rendering your project

4.1.1 Rendering from VSCode

You can render your project by:

1. Open a `.qmd` file (e.g. `index.qmd`) in VSCode.
2. Open the command palette (Ctrl-shift-P) and begin typing `Quarto: Render Project`
3. Select what to render: `All formats | HTML | PDF`

Tip

Alternatively, you can launch a live preview which automatically updates as you edit your content. Launch Quarto: Preview or **Ctrl-Shift-K**

4.1.2 Rendering from the command line

You can either use the Terminal in VSCode (or just have a terminal open) and render the document from a terminal using:

```
quarto render
```

Tip

Alternatively, you can launch a live preview which automatically updates as you edit your content.

```
quarto preview
```

4.1.3 Quick preview

For a quick preview of simple markdown code, the **Markdown All in One** VSCode extension, will show an instant preview of any single markdown document. You can run this Preview from the VSCode command palette of **Ctrl-Shift-V**

Warning

Please note the quick preview is just a simple markdown display preview (it won't know about references, figures, code) and doesn't rebuild your whole project, but can be useful for quick previews to check things look *roughly* right.

Note

Congratulations, you've built your first Quarto project! In the following chapters we will learn how to add content.

5 Basic Markdown

Most Quarto documents are written using Markdown.

Markdown is a lightweight syntax for adding structure and formatting to plain text.

5.1 Headings

```
# Heading 1
```

```
## Heading 2
```

```
### Heading 3
```

5.2 Bold and Italics

```
*italic text*
```

```
**bold text**
```

5.3 Lists

Unordered list:

- Item one
- Item two
- Item three

Ordered list:

1. First item
2. Second item
3. Third item

5.4 Links

You should always create [clear descriptive links](https://teachinghub.bath.ac.uk/guide/creating-meaningful-links/).

You should always create [clear descriptive links](#).

5.5 Basic Images

Basic images with alt text are created with:

```
![Figure caption:here are some dice](images/dice.png){fig-alt="4 colourful  
dice tumbling mid air"}
```



Figure 5.1: Figure caption:here are some dice

We will cover how to include figures that you can reference, caption etc in [Chapter 7](#).

6 Converting Existing LaTeX Notes

Fortunately, a large proportion of LaTeX mathematics can be copied directly into Quarto documents without modification.

For example:

```
\[  
F = ma  
\]
```

and

```
\begin{equation}  
E = mc^2  
\end{equation}
```

typically work immediately.

6.1 Recommended Migration Approach

Rather than converting an entire module at once:

1. Choose a single lecture.
2. Copy content into a .qmd file.
3. Render frequently.
4. Fix issues incrementally.

Common areas that may need attention include:

- TikZ diagrams
- Custom macros
- Complex tables
- Package-specific formatting

6.2 Automatically converting LaTeX

Pandoc can convert between lots of different formats including LaTeX (.tex) and Quarto (.qmd). **It won't do a perfect job** and which is why it's recommended to carefully go through your TeX source. However if your TeX is *relatively clean* (few macros, extra packages, complex float layouts), then Pandoc can get you going quite quickly.

From the command line you can run

```
pandoc -f latex -t markdown input-tex.tex -o output-md.qmd
```

! Important

You will need to review the output .qmd to make sure you have output that is compatible.

🔥 Caution

Pandoc is not aware of your whole TeX project: you need to convert each .TeX separately.

6.3 Tidying up the .qmd file

Here are some things Pandoc does not readily convert into markdown:

- Section etc labels - if you used underscores in any of your latex labels you should change these to '-' in the markdown file. The underscores confuse markdown. Chapter 10 discusses sections, chapters, etc.
- Figures - the default pandoc conversion is quite limited, so here we'll use a more adaptable figure environment. In practice its easiest to just copy-paste the example code in Chapter 7.
- Equations - most equations convert fine but if you want to have equation numbers and labels you'll need to do some tidying. See Chapter 8 for the syntax.
- Tables - tables have a different format but it's pretty easy to convert a latex table using find/replace or online table editors/converters (e.g. [Table Convert](#))
- References - generally easy to fix - see Chapter 11

This list is based on the [Making Lecture Notes with Bookdown guide by Vicky Scowcroft](#)

7 Figures

💡 Tip

For full details and advanced options, see the [official Quarto Figures documentation](#):

7.1 Basic Figure

The example below creates:

- a numbered figure,
- a visible caption,
- alternative text for screen readers,
- a cross-reference target.

```
! [Figure caption] (images/dice.png) {#fig-dice fig-alt="This is descriptive alt text"}
```

```
@fig-dice shows four multi-coloured dice.
```

Figure 7.1 shows four multi-coloured dice.

💡 Tip

In Quarto, an image becomes a figure whenever it has a caption and appears by itself in a paragraph.



Figure 7.1: Figure caption

7.2 Layout Options

7.2.1 Size

```
! [Figure caption] (images/dice.png) {#fig-dice fig-alt="This is descriptive alt text" width=
! [Figure caption] (images/dice.png) {#fig-dice fig-alt="This is descriptive alt text" width=
```



Figure 7.2: Figure caption



Figure 7.3: Figure caption

7.2.2 Subfigures

Subfigures allow multiple related images to be grouped under a single figure number with individual sub-captions.

```
::: {#fig-dice-layout layout-ncol=2}
! [Figure caption for left image] (images/dice.png) {#fig-dice-left fig-alt="This is descript
! [Figure caption for right image] (images/dice.png) {#fig-dice-right fig-alt="This is descri
Here is the main caption for the whole figure.
:::
```

Notes:

- Individual image captions become sub-captions.
- The final paragraph becomes the overall figure caption.
- **Blank lines between images and captions are required.**



(a) Figure caption for left image



(b) Figure caption for right image

Figure 7.4: Here is the main caption for the whole figure.

- You can also create more custom layouts: `{layout-ncol=3}`, `{layout-nrow=2}`

7.2.3 Two Images Above One Wide Image

i Note

For brevity in the next few examples, I have just used pseduocode `{{image.png}}` to show where the image should be

```

::: {layout="[[1,1],[1]]"}

{{image1.png}}

{{image2.png}}

{{image3.png}}

:::

```

7.2.4 Unequal Widths

```
::: {layout="[[70,30],[100]]"}  
  
{{large.png}}  
  
{{small.png}}  
  
{{full.png}}  
  
:::
```

7.2.5 Adding Space Between Elements

```
::: {layout="[[40,-20,40],[100]]"}  
  
{{left.png}}  
  
{{right.png}}  
  
{{bottom.png}}  
  
:::
```

Negative values create spacing between items.

7.3 Figures from code

Figures generated from Python, R, or Julia code are **automatically** treated as figures. See Section [9.5](#) for examples

7.4 Figure Divs

Any content can be treated as a figure by enclosing it in a fenced div with an identifier beginning with `fig-`. This is useful for:

- embedded videos
- interactive visualisations
- HTML widgets
- custom content that is not a simple image

! Important

These only work for the HTML output and not PDF or Word. If you include a figure div, you should also include a fallback link to the resource in the caption to support the other output formats.

```
::: {#fig-video}
```

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/EbAAmrB0luA?si=jYVDsTR
```

The final paragraph becomes the figure caption. You should include [link to the video here

```
:::
```

Figure 7.5: The final paragraph becomes the figure caption. You should include [link to the video here](#)

7.5 Tikz figures

⚠ TO DO

Include link to Tikz examples.
See [quarto_tikz extension](#)

8 Maths Equations

Quarto uses MathJax for mathematics in HTML outputs.

8.1 Inline Mathematics

The force is given by `$F = ma$`.

Produces:

The force is given by $F = ma$.

8.2 Display Mathematics

```
$$  
F = ma  
$$
```

8.3 Numbered Equations

Numbered equations can be labelled.

```
$$  
F = ma  
$$ {#eq-newton}
```

$$F = ma \tag{8.1}$$

8.4 Aligned equations

You can use the `\begin{align}` or `\begin{aligned}` to align equations (note `\begin{eqnarray}` also works, but is generally deprecated and *not recommended for TeX nowadays*)

```
$$
\begin{align}
a &= b + c \\
&= d + e
\end{align}
$$
```

$$a = b + c \tag{8.2}$$

$$= d + e \tag{8.3}$$

Caution

MathJax can't have multiple labels for a multi-line equation (as of Oct 2026) - you can only label the whole `align` environment block. There are various workarounds: the simplest and least technical is to have separate equations (but lose the align control). It is still possible to label the whole block but this doesn't present nicely (again more workarounds are possible, but this is *MathJax* rather than Quarto limitation.)

```
$$
\begin{aligned}
\nabla \cdot \mathbf{E} &= \frac{\rho}{\epsilon_0} \\
\nabla \cdot \mathbf{B} &= 0 \\
\nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t} \\
\nabla \times \mathbf{B} &= \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \frac{\partial \mathbf{E}}{\partial t}
\end{aligned}
$$ {#eq-maxwell}
```

$$\begin{aligned}
 \nabla \cdot \mathbf{E} &= \frac{\rho}{\varepsilon_0} \\
 \nabla \cdot \mathbf{B} &= 0 \\
 \nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t} \\
 \nabla \times \mathbf{B} &= \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t}
 \end{aligned}
 \tag{8.4}$$

8.5 Referencing Equations

Once labelled, equations can be referenced using:

`Equation @eq-newton` describes the relationship between force, mass and acceleration.

Equation [Equation 8.1](#) describes the relationship between force, mass and acceleration.

8.6 Newcommands and macros

Warning

To do: include a simple way to create custom maths macros for HTML/PDF outputs.

9 Code

Quarto supports executable code, code examples, and syntax-highlighted snippets for a wide range of programming languages. This makes it ideal for creating teaching materials, laboratory manuals, lecture notes, and technical documentation.

9.1 Including inline code snippets

You can include inline code snippets in between single backticks, e.g. ``print("Hello, world!")`` (this renders as `print("Hello, world!")`).

9.2 Including code blocks

You can include code blocks in triple backticks, followed by the language name (for syntax highlighting). For example:

```
```python
print("Hello, world!")
```
```

This results in the following rendered code output with syntax highlighting:

```
import numpy as np
print("Hello, world!")
```

Tip

Supported languages for syntax highlighting include: Python, R, Bash, C, C++, Java, JavaScript, MATLAB, markdown and many more. See the [Quarto documentation](#) for a full list of supported languages.

For completely unformatted text use the ````text` option.

9.3 Running Code

Executable code is placed inside a code chunk (**notice the curly braces after the opening triple backticks**). You can specify the programming language for the code chunk (e.g. `python`, `r`, `julia`, `bash`, etc.) and Quarto will automatically highlight the syntax.

```
```{python}

print("Hello, world!")

```
```

This results in the following rendered output (shows code and output):

```
print("Hello, world!")
```

Hello, world!

9.4 Options for running code

You can control how code is executed and displayed using chunk options. For example, you can hide the code but show the output, or hide both the code and the output. You can also control whether warnings and messages are displayed.

You can set chunk options for individual code chunks using the `#|` syntax directly after the opening triple backticks.

9.4.1 Showing Output but Hiding Code

You can run the code and display the output, **but hide the code source from viewers** with `#| echo: false`

```
```{python}

#| echo: false

print("Only the output will appear.")

```
```

This results in the following rendered output:

Only the output will appear.

9.4.2 Folding Long Code Blocks

Long code examples can be collapsed to avoid cluttering the document with `#| code-fold: true`. You can also add a summary to the code block with `#| code-summary: "Show Python code"`.

```
```{python}
#| code-fold: true
#| code-summary: "Show Python code"
print("This code is initially hidden.")
...

```

This results in the following rendered output:

```
print("This code is initially hidden.")
```

This code is initially hidden.

### Note

The folding code blocks are expanded in the static PDF output.

### Tip

It's a good idea to keep code lines short (e.g. < 80 characters) as the LaTeX output does not wrap long lines of code.

## 9.5 Figures from code

Figures generated from Python, R, or Julia code are **automatically** treated as figures. To make these accessible, you should include:

- `#| label: fig-line-plot` - this gives a reference label to the figure
- `#| fig-cap: "A simple line plot"` - this gives a figure caption
- `#| fig-alt: "Descriptive alt text"` - this gives alternative text for accessibility (e.g. screen readers)

For example, the following code chunk generates a simple line plot:

```
```{python}
#| label: fig-line-plot
#| fig-cap: "A simple line plot"
#| fig-alt: "Line graph showing values increasing and decreasing across four
observations."

import matplotlib.pyplot as plt
plt.plot([1, 23, 2, 4])
plt.show()
```
```

This results in the following rendered output:

```
import matplotlib.pyplot as plt

plt.plot([1, 23, 2, 4])
plt.show()
```

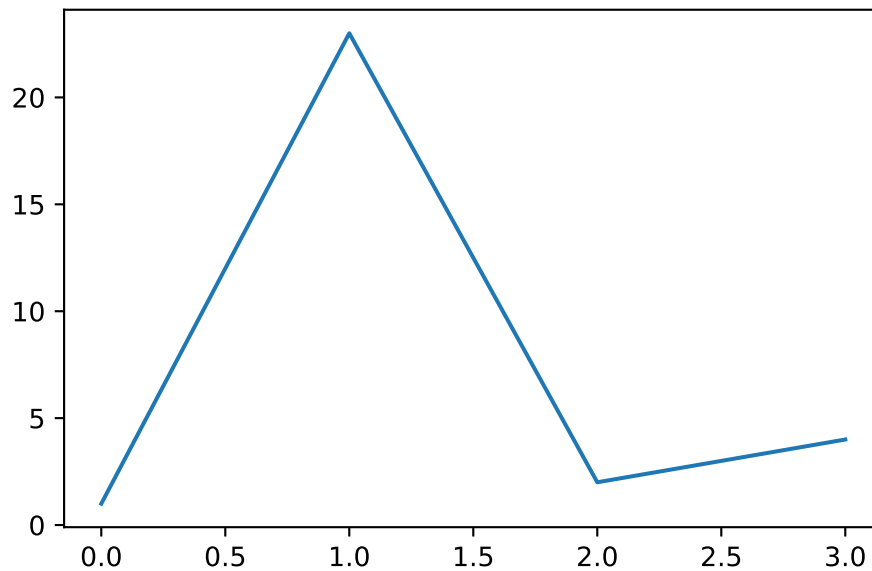


Figure 9.1: A simple line plot

### 💡 Tip

Often it's useful to add `#!/ echo: false` at the start of the figure will make sure to not echo the code and only display the figure:

```
```{python}
#!/ echo: false
#!/ label: fig-line-plot-noecho
#!/ fig-cap: "A simple line plot"
#!/ fig-alt: "A simplified plot with the#!/ echo: false to not show the code"
import matplotlib.pyplot as plt
plt.plot([1, 23, 2, 4])
plt.show()
```
```

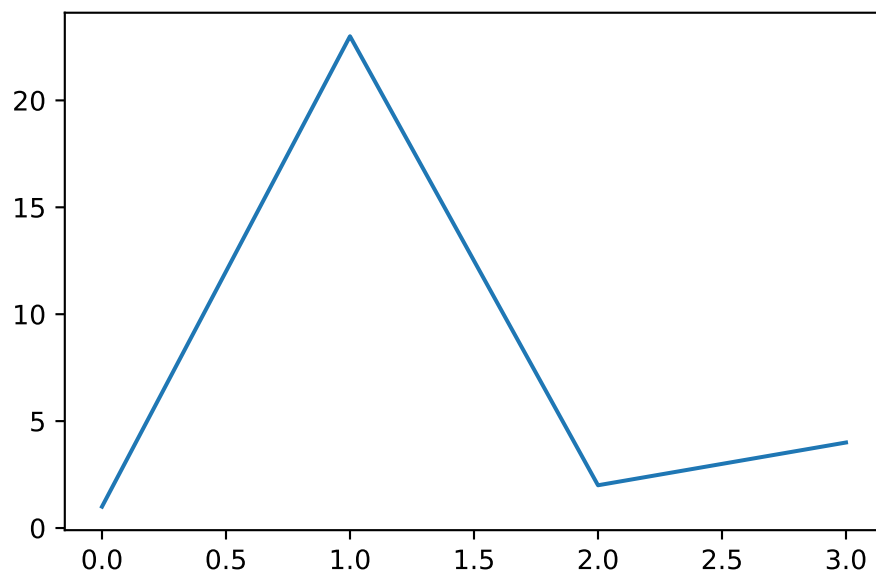


Figure 9.2: A simple line plot

## 9.6 Global Code Options

Chunk options can be applied to every code block using the document YAML.

```

execute:
 echo: false
```

```
warning: false
message: false

```

# 10 Working with Sections

Section headings create document structure automatically.

```
Introduction

Background

Methods

Results

Conclusions
```

Quarto can automatically number sections using:

```
format:
 html:
 number-sections: true
```

## 10.1 How sections are displayed

Each separate .qmd file will appear in the left hand navigation menu as a (numbered) chapter.

Within each .qmd, each heading level will appear in the table of contents (right hand side)

- Heading level 1 `#` will increment the number (e.g. 1,2,3 etc)
- Heading level 2 `##` will increment the subpoints (e.g. 1.1, 1.2, 1.3 etc)
- Heading level 3 `###` will increment the sub-subpoint (e.g. 1.1.1, 1.1.2, 1.1.3 etc)

## 10.2 Numbering sections

Sections are numbered by default (`# Section heading`). You can create a unnumbered heading with `{.unnumbered}` (e.g. `)# Unnumbered section heading {.unnumbered}`)

### 10.2.1 Global defaults

Numbered sections are the default and can be controlled in the yaml

```
format:
 html:
 number-sections: true
 toc: true
 toc-depth: 3
 toc-location: right
 pdf:
 number-sections: true
```

## 10.3 Labelling sections

Sections can be assigned identifiers.

```
Newton's Laws {#sec-newton}
```

#### ! Important

You should use – rather than \_ in section labels as underscores can confuse Quarto!

## 10.4 Referencing sections

You can then reference the section elsewhere:

```
See @sec-newton for an introduction to Newton's Laws.
```

When rendered, Quarto automatically inserts the appropriate section number.

# 11 References and Citations

Quarto supports citation management through BibTeX and a .bib file in the normal way.

## 11.1 Connecting the Bibliography

Add the bibliography file to your YAML:

```

bibliography: references.bib
csl: harvard-university-of-bath.csl

```

### Note

A copy of the [Harvard Bath style file](#) is included in these notes.

##Citing Sources

Citations can be added using citation keys.

A single citation for literate programming @knuth84.

A single citation for literate programming Knuth (1984).

Multiple citations:

Multiple citations can be cited like this [@knuth84; @le2026quarto].

Multiple citations can be cited like this (Knuth 1984; Le Foll 2026).

## 11.2 Reference list

A reference list is generated automatically at the end of each chapter page.

If you prefer one single reference list for the whole book, include a separate chapter with a specific call to the reference list:

```
References {.unnumbered}
::: {#refs}
:::
```

# 12 More Markdown

## 12.1 Tables

Simple tables can be written directly in Markdown.

```
Quantity	Symbol	SI Unit
Force	F	N
Energy	E	J
Mass	m	kg
```

### 12.1.1 Labelled Tables

Tables can also be labelled for cross-reference.

```
Quantity	Symbol	SI Unit
Force	F	N
Energy	E	J
Mass	m	kg
```

```
: SI units used throughout this module {#tbl-siunits}
```

Table 12.1: SI units used throughout this module

| Quantity | Symbol | SI Unit |
|----------|--------|---------|
| Force    | F      | N       |
| Energy   | E      | J       |
| Mass     | m      | kg      |

Reference the table using:

See `@tbl-siunits` for a summary of units.

See Table 12.1 for a summary of units.

## 12.2 Collapsible Sections

```
<details>
<summary>Show solution</summary>
```

The solution can be placed here.

```
</details>
```

Show solution

The solution can be placed here.

## 12.3 Callout highlighted Boxes

You can include callout boxes to format text areas to draw attention to certain parts. There are five different environments and a custom environment

- note
- tip
- important
- caution
- warning

```
::: {.callout-note}
```

Here is a callout note.

```
:::
```

**i** Note

Here is a callout note.



Tip

Here is a callout tip.



Important

Here is a callout important.



Caution

Here is a callout caution.



Warning

Here is a callout warning.

```
... {.callout-note}
Custom title
```

Here's my custom block, which displays like a note!

```
...
```



Custom title

Here's my custom block, which displays like a note!

## 12.4 Footnotes

### 12.4.1 Inline footnotes

Footnotes can be created with square brackets.<sup>1</sup>

Footnotes can be created with square brackets.<sup>1</sup>[This is a footnote]

---

<sup>1</sup>This is a footnote

## 12.4.2 Defining footnotes

For longer documents, you may prefer to define footnotes separately:

```
Quarto supports footnotes.[^1]
```

```
[^1\]: This is a footnote.
```

### Tip

Use footnotes for supplementary information rather than essential content. Avoid overusing footnotes, as they can distract readers from the main narrative and make your content less accessible to process!

## 12.5 Hiding/commenting out content

You can also include HTML comments that will not be visible on the page.

### Warning

These HTML comments are still visible in the rendered HTML source (this is very different from LaTeX). **Please don't put anything there that you don't want students to see (e.g. solutions!)**

```
<!-- Update this example before publication -->
```

These can also span multiple lines (which can be useful for commenting out sections you don't need!)

```
<!--
TODO:
- Add an example for cross-references.
- Check accessibility guidance.
- Review screenshot captions.
-->
```

## 12.6 Quarto extensions

Quarto extensions are a way to add new features to Quarto. They can be used to create new output formats, add new functionality, or customize the behavior of Quarto.

See [the Quarto extensions guidance](#) for a list of extensions and how to create your own.

# 13 Tweaking the YAML Header

The YAML block appears at the top of every Quarto document.

A simple example:

```

title: "Introduction to Fluid Mechanics"
author: "Dr Jane Smith"
date: today

```

## 13.1 Table of Contents

Enable a table of contents:

```
format:
 html:
 toc: true
```

## 13.2 Section Numbering

```
format:
 html:
 number-sections: true
```

## 13.3 Themes

Quarto provides several built-in themes.

```
format:
 html:
 theme: cosmo
```

Other popular themes include:

- flatly
- litera
- united
- lumen
- sandstone

If you specify a light and dark theme, then Quarto will add in a toggle button to allow users to switch.

```
format:
 html:
 theme:
 light: cosmo
 dark: darkly
```

#### Caution

If you use dark mode, please if you images have transparency (if you have black text on a transparent background this can be illegible).

# 14 Multiple Output Formats

Quarto can render several formats from the same source.

```
format:
 html:
 toc: true
 pdf:
 toc: true
 docx: default
```

Rendering the document then produces HTML, PDF, Word simultaneously.

## Caution

Always check each output. Sometimes things may not always compile as you wish!

# 15 Creating Books and Lecture Notes

A common approach is to keep all chapter files in a dedicated chapters folder and store chapter-specific images in an images directory within that folder.

```
project:
 type: book
```

Example structure:

```
course-notes/

 _quarto.yml
 index.qmd
 references.bib

 chapters/
 week1.qmd
 week2.qmd
 week3.qmd

 images/
 lecture-diagram.png
 experiment-setup.jpg
 results-plot.svg
```

Example `_quarto.yml`:

```
project:
 type: book

book:
 title: "Physics 1A Lecture Notes"

chapters:
- index.qmd
```

- chapters/week1.qmd
- chapters/week2.qmd
- chapters/week3.qmd

# 16 Publishing a Quarto Project

Once you have created your Quarto project, you will need to publish the rendered output so that others can access it. Quarto supports several publishing options, but for most teaching-related content at the University of Bath, publishing to Moodle is the recommended approach.

## 16.1 Publish to Moodle

For teaching materials intended for University of Bath students, we recommend publishing your Quarto website or mini-site through Moodle.

Benefits include:

- Available directly to students through Moodle.
- Included within the University's Moodle archive processes.
- Covered by established institutional retention arrangements.
- Access controlled through Moodle course permissions.
- No requirement for students to visit external websites.

Quarto websites generate a collection of HTML, CSS, JavaScript, image, and supporting files. These can be hosted within Moodle as a self-contained mini-site.

### 16.1.1 Moodle publishing workflow

1. Render your Quarto project.
2. Locate the generated `_site` or output folder.
3. Create a ZIP archive containing the entire published site and all supporting files.
4. In Moodle, create a **File** activity (not a Folder activity).
5. Upload the ZIP file and use Moodle's **Unzip** option.
6. Set the site's `index.html` file as the **Main File**.
7. Save the activity and test that the site launches correctly.

The University of Bath Moodle currently has a 50 MB upload limit for file activities. If your published site exceeds this size please contact [tel@bath.ac.uk](mailto:tel@bath.ac.uk)

**! Important**

Detailed step-by-step guidance is available at [Publishing an HTML mini-site to Moodle](#)

### 16.1.2 Updating a Moodle-hosted site

When you make changes to your Quarto project:

1. Re-render the site locally.
2. Create a new ZIP archive of the updated output.
3. Replace the all existing files in Moodle.
4. Ensure the correct `index.html` file remains set as the Main File.

## 16.2 Public Resources: GitHub Pages

If your Quarto content is intended for public access, GitHub Pages is an excellent option.

Advantages include:

- Free hosting.
- Integrated version control through Git.
- Collaborative authoring workflows.
- Automatic publishing when content is updated.
- Suitable for open educational resources, documentation, and public-facing guides.

GitHub Pages is particularly useful for resources that do not need Moodle authentication and are intended to be shared publicly.

# References

- Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* (USA) 27 (2): 97–111.  
<https://doi.org/10.1093/comjnl/27.2.97>.
- Le Foll, Elen. 2026. *Quarto for Reproducible Research Workflows and Academic Publishing*.  
<https://elenlefol.quarto.pub/quarto4research/Quarto4Research.pdf>.

# **Part I**

## **Appendices**